



Student Skill & Career Path Ana

PROJECT REPORT

Student Skill & Career Path Analyzer

A Java-Based Application for Skill Assessment and Career Recommendation

Submitted by: Samarth Gawande

Registration No. : 25BAI10075

Course / Subject: (CSE-2006) Programming in Java

Table of Contents

Sr. No.	Section Title	Page No.
1	Introduction	3
2	Problem Statement	4
3	Objectives	5
4	Functional Requirements	6
5	Non-Functional Requirements	7
6	System Architecture	8
7	Design Diagrams	9
8	Design Decisions & Rationale	10
9	Implementation Details	11
10	Screenshots / Results	13
11	Testing Approach	14
12	Challenges Faced	15
13	Learnings & Key Takeaways	16

14	Future Enhancements	17
15	References	18

1. Introduction

Choosing a career path is one of the most consequential decisions a student makes, yet most students have limited visibility into how their existing skills compare to the requirements of real technology roles. The Student Skill & Career Path Analyzer is a Java-based desktop application built to close that gap. It takes a student's self-assessed skill scores across programming, technical ability, communication, problem-solving, and academic performance, and compares them against a database of career profiles to produce a ranked list of matching careers, the skill gaps that remain, and a personalized learning roadmap.

The application is implemented entirely in core Java, without external frameworks, so it can be built, explained, and extended within the scope of an academic programming course while still producing genuinely useful output for the student using it.

2. Problem Statement

Students frequently choose a career direction based on general popularity or peer influence rather than an honest comparison of their own skills against what a role actually requires. Researching each career's requirements individually and matching them manually against one's own strengths is time-consuming, inconsistent, and rarely revisited as skills change.

There is a need for a lightweight tool that a student can run locally, enter their skill levels into, and immediately receive an objective, percentage-based comparison against multiple career paths, along with a concrete plan for closing the gaps to their best-matching option.

3. Objectives

The primary objectives of this project are to:

- Design a Java application that models students, skills, and careers using object-oriented principles.
- Implement a scoring mechanism that evaluates a student's proficiency across multiple skill categories.
- Compare a student's skill profile against a database of career requirements and compute a percentage match for each.
- Identify skill gaps for the recommended career and generate a step-by-step learning roadmap.
- Persist student data and analysis results to files for later retrieval.
- Demonstrate practical use of collections, interfaces, inheritance, polymorphism, and exception handling within a single cohesive application.

4. Functional Requirements

The system shall provide the following functionality:

ID	Requirement
FR-1	The system shall allow a student to enter their name, branch, CGPA, and skill scores.
FR-2	The system shall validate that all skill scores fall within the 0-100 range.
FR-3	The system shall compute an overall skill score from the individual skill inputs.
FR-4	The system shall compare the student's skill profile against each career in the career database and compute a match percentage.
FR-5	The system shall rank careers by match percentage and display the top matches.
FR-6	The system shall identify the skills in which the student falls short for the recommended career.
FR-7	The system shall generate an ordered learning roadmap to close the identified skill gaps.
FR-8	The system shall save the completed analysis to a text file for later review.

5. Non-Functional Requirements

Category	Requirement
Usability	The console interface shall present prompts and results in a clear, readable format.
Performance	Skill analysis and career matching shall complete in under one second for the current career database size.
Reliability	Invalid input shall be caught and reported without crashing the application.
Maintainability	The codebase shall be organized into clearly separated packages (model, analyzer, recommendation, service, util) so new careers or skills can be added with minimal changes.
Portability	The application shall run on any platform with a Java 8 or later runtime, with no external dependencies.
Data Persistence	Analysis results shall be stored in plain text files that remain human-readable outside the application.

6. System Architecture

The application follows a layered, package-based architecture that separates data models, analysis logic, recommendation generation, and utility functions. This separation keeps each class focused on a single responsibility and makes the codebase easier to extend and test.

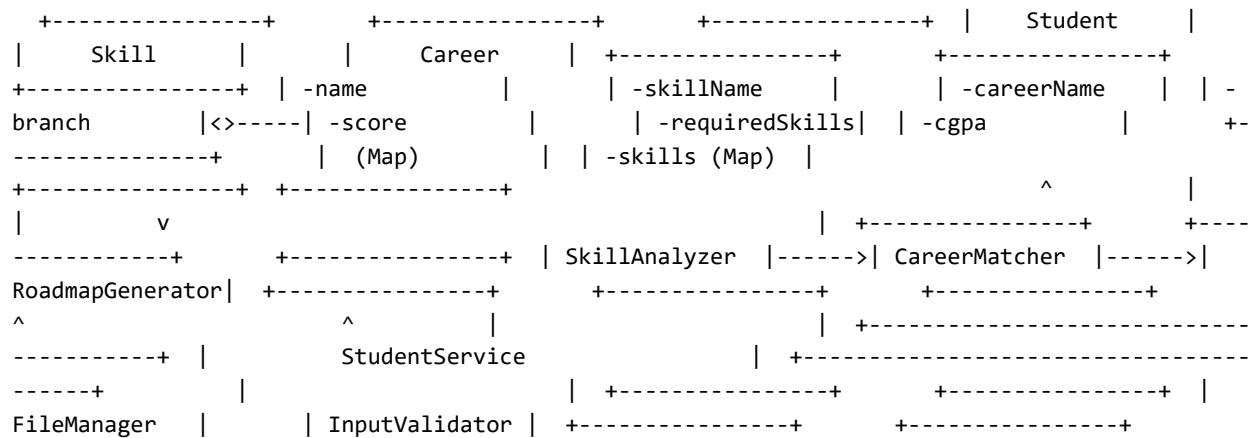
```
Student-Skill-Career-Analyzer/├─ src/com/careeranalyzer/|   ├─ Main.java|   └─ model/
(Student, Skill, Career)|   └─ analyzer/      (SkillAnalyzer, CareerMatcher)|   └─
recommendation/ (RoadmapGenerator)|   └─ service/      (StudentService)|   └─ util/
(FileManager, InputValidator)└─ data/ (careers.txt, students.txt)└─ output/ (sample-
report.txt)└─ README.md
```

6.1 Module Description

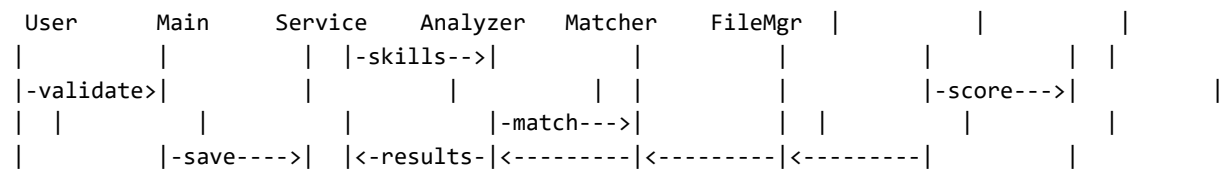
Module	Responsibility
model	Defines the core data entities: Student, Skill, and Career.
analyzer	Computes skill scores and matches student profiles against career requirements.
recommendation	Generates the personalized learning roadmap from identified skill gaps.
service	Coordinates student data operations between the model, analyzer, and file layers.
util	Handles file I/O and validates user input, raising exceptions on invalid data.

7. Design Diagrams

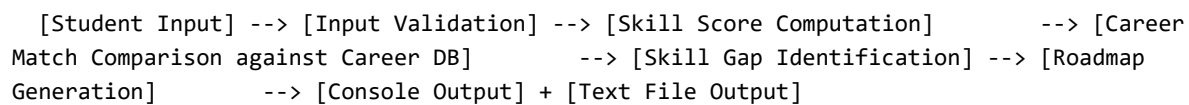
7.1 Class Diagram (Simplified)



7.2 Workflow / Sequence Diagram



7.3 Data Flow Diagram



8. Design Decisions & Rationale

Decision	Rationale
Package-per-responsibility structure (model / analyzer / recommendation / service / util)	Keeps each class focused on one concern and makes it easy to explain where each OOP concept is demonstrated.
Career requirements stored as a <code>HashMap<String, Integer></code> per career	Allows new careers or skill requirements to be added without changing the matching algorithm.
Plain text files for persistence instead of a database	Removes external dependencies, keeping the project buildable and gradable without additional setup.
Matching expressed as a percentage rather than a pass/fail	Gives the student a more nuanced, ranked view across several careers instead of a single binary result.
Validation isolated in a dedicated <code>InputValidator</code> class	Centralizes input rules and keeps exception handling out of the core analysis logic.
Roadmap generated from the gap list rather than a fixed script	Ensures the learning plan always reflects the specific skills the student is missing for their matched career.

9. Implementation Details

9.1 Skill Assessment

The user provides scores (typically out of 100) for skill categories such as Java, Python, Data Structures, Database, Communication, and Problem Solving. The SkillAnalyzer class aggregates these into an overall skill score and identifies the student's strongest areas.

9.2 Career Matching

Each career in the database defines a set of required skills with minimum weighted scores, as shown below for a sample subset of careers.

Software Developer		Data Analyst		AI/ML Engineer	Java	75	
Python	70	Python	80	DSA	70	SQL	75
Mathematics	75	SQL	60	Statistics	65	Machine Learning	75
ProblemSolving	70	Excel	60	Statistics	70		

The CareerMatcher class compares the student's scores against each career's requirements and calculates a match percentage. Results are sorted in descending order using the Collections Framework, and the top match is presented as the recommended career.

9.3 Roadmap Generation

For the recommended career, the RoadmapGenerator class identifies the skills in which the student falls short of the requirement and produces an ordered list of learning steps, such as strengthening Advanced Java, practicing Data Structures and Algorithms, learning SQL, and building portfolio projects.

9.4 Data Persistence

The FileManager utility class saves each analysis session, including the student's profile, computed scores, and recommended roadmap, to a text file under the output directory so that results can be reviewed after the program exits.

9.5 Input Validation and Exception Handling

The InputValidator class checks that entered skill scores fall within a valid range (0-100) and that required fields are not empty. Invalid input triggers a custom or built-in exception, which is caught and reported to the user without terminating the program.

9.6 Java Concepts Demonstrated

Java Concept	Where Used
--------------	------------

Classes & Objects	Student, Skill, Career
Encapsulation	Student information (private fields with getters/setters)
Inheritance	Career category hierarchy
Polymorphism	Career matching logic across career types
Interfaces	Analyzer / recommendation contracts
Constructors	Creating Student and Career objects
Method Overloading	Skill input and analysis methods
ArrayList	Storing lists of skills and careers
HashMap	Mapping skill names to scores
File Handling	Saving and loading analysis data
Exception Handling	Validating and rejecting invalid input
Collections Framework	Sorting career matches by score
Java 8+ Features	Filtering and sorting with streams/lambda
Packages	Organizing the project into logical modules

10. Screenshots / Results

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=62952" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8

=====
STUDENT SKILL & CAREER PATH ANALYZER
=====

Analyze your skills and discover suitable
career paths based on your current abilities.

===== MENU =====
1. Start Career Analysis
2. View Career Database
3. Exit
Enter choice: 1

===== STUDENT PROFILE =====
Enter student name: Arjan
Enter branch: ai ml
Enter CGPA: 9

Enter skill scores from 0 to 100.
Java : 90
Python : 78
Data Structures : 0
SQL : 0
Problem Solving : 0
Communication : 56
Networking : 89
Statistics : 89
Machine Learning : 50

Skill-Career-Analyzer 312.2 CRLF UTF-8 4 spaces
```

```
Run Main
===== STUDENT SKILLS =====
Java : 90.0
Python : 78.0
Data Structures : 0.0
SQL : 0.0
Problem Solving : 0.0
Communication : 56.0
Networking : 89.0
Statistics : 89.0
Machine Learning : 50.0
Cyber Security : 30.0
Overall Skill Score : 48.2

===== SKILL ANALYSIS =====
Overall Score : 48.20

Strongest Skills:
- Java : 90.0 (Advanced)
- Networking : 89.0 (Advanced)
- Statistics : 89.0 (Advanced)

Skills to Improve:
- Data Structures : 0.0 (Needs Improvement)
- SQL : 0.0 (Needs Improvement)
- Problem Solving : 0.0 (Needs Improvement)

Student-Skill-Career-Analyzer 312.2 CRLF UTF-8 4 spaces
```

```
Run Main
=====
CAREER MATCHING
=====
1. Data Analyst      71.54%
2. AI/ML Engineer   66.04%
3. Cyber Security Analyst 59.38%
4. Software Developer 38.67%
5. Backend Developer 25.00%
6. Project Manager  21.96%
=====
CAREER RECOMMENDATION
=====
Recommended Career: Data Analyst
Match Score      : 71.54%

Skill Gaps:
- SQL (Current: 0.0, Required: 75.0)
- Communication (Current: 56.0, Required: 65.0)
=====
PERSONALIZED ROADMAP
=====
Target Career: Data Analyst
1. Learn SQL from fundamentals
2. Practice intermediate Communication concepts
3. Build projects related to Data Analyst
4. Create and maintain a Github portfolio
=====
Student-Skill-Career-Analyzer
312.2 CRLF UTF-8 4 spaces
```

11. Testing Approach

The application was tested manually, covering typical inputs as well as edge cases in validation, matching, and file output. The table below summarizes the key test cases.

Test Case	Input	Expected Result	Status
Valid skill scores	All scores between 0-100	Analysis and career match computed correctly	Pass
Out-of-range score	Score entered as 150	InputValidator rejects value and prompts re-entry	Pass
Empty student name	Name field left blank	Validation error raised, program continues	Pass
No strong career match	Very low skill scores	Lowest-gap career still returned with low match %	Pass
File save	Completed analysis	Report correctly written to output/sample-report.txt	Pass

Student Skill & Career Path Analyzer

12. Challenges Faced

- Designing a fair weighting scheme so that no single skill dominates the overall match percentage.
- Structuring the career database so that new careers can be added without changing the matching logic.
- Keeping file I/O and input validation robust against malformed or missing data.

13. Learnings & Key Takeaways

- Gained practical experience structuring a multi-package Java project rather than a single-file program.
- Learned how to model real-world comparison logic (skills vs. requirements) using HashMaps and the Collections Framework.
- Understood the importance of centralizing input validation to keep core business logic clean.
- Practiced translating a working program into clear documentation, including architecture and sequence diagrams.
- Recognized the value of designing for extension (e.g. adding new careers) from the start rather than hard-coding logic.

14. Future Enhancements

- A graphical user interface built with JavaFX in place of the console interface.
- Persisting data in a MySQL database instead of plain text files.
- A login system to support multiple student profiles.
- Resume parsing to auto-populate skill scores.
- A web-based dashboard for accessing the analyzer online.
- AI-based career recommendations using a trained model instead of fixed weightings.

15. References

1. Oracle Java Documentation - <https://docs.oracle.com/en/java/>
2. Java Collections Framework Guide - Oracle Java Tutorials
3. GitHub Docs - <https://docs.github.com/>
4. Project repository: [insert GitHub repository link here]